

Kaspar

A turn-key CME futures trading system that is also the order book simulator you backtest it in.

VENUE
CME Globex

MARKET DATA
MDP3 SBE

ORDER ENTRY
iLink 3

BOOK
MBP & MBO L3

LICENCE
MIT, open source

Kaspar is two things sharing one codebase. It is a production trading system — multicast market data in, full order books reconstructed order by order, an execution algorithm on top, and authenticated iLink 3 sessions out to the exchange. It is also a market simulator: the same books rebuilt from recorded packet captures, with your orders sitting in the price-time queue and filling only when the market actually trades through them.

The same strategy code, the same execution algorithm and the same book run in PCAP replay, in live paper trading, and against the live exchange. You move between them by changing configuration, not by porting anything — so a backtest exercises the code path that will trade.

WHAT IS IN THE BOX

No vendor feed handler, no broker API, no assembly required.

MARKET DATA

Full **MDP3 SBE** decoder: incremental book updates, trades, order-by-order MBO, instrument definitions, snapshot recovery. Sequence gaps are detected and repaired automatically.

ORDER BOOK

Order-by-order reconstruction holding **every resting order individually**, not aggregated depth — which is what makes a queue position, and therefore a fill inference, definable at all.

ORDER ENTRY

iLink 3 session handler: SBE encoding, HMAC authentication, sequence management, negotiation and establishment, primary/secondary failover.

EXECUTION

Shadow-POV, a passive/aggressive percentage-of-volume algorithm that places by following orders in the market. Measured over a full calendar year — see below.

SIMULATION

Three modes off one binary: **PCAP replay**, live multicast paper trading, live iLink execution. Deterministic — a fixed session replays to identical fills.

CONCURRENCY

A custom **C++20 actor framework** — a proven concurrency model, and the same code whether the hot path runs across threads or entirely on one. Scale out by giving actors their own threads; collapse to a single-threaded hot path by grouping them, with no rewrite either way.

STRATEGY AUTHORIZING IN C++ OR RUST

C++ in-process actors for the lowest latency, or Rust across an in-process FFI interop — no socket, no serialisation between the strategy and the book.

MEASUREMENT

An execution-cost harness that runs the algorithm as a two-sided maker over recorded sessions, one row per window, plus a parameter sweep over rate, size and simulated latency.

PERFORMANCE

AMD EPYC 9374F · RHEL 9 · g++ 15 · -O3 -march=native

The usual objection to actors is messaging overhead. The answer is `fast_send`, which runs the receiver's handler inline on the caller's thread and returns the reply as a value.

PATH	P50	WHAT IT COSTS
<code>fast_send</code> — inline, no thread hop	~10 ns	≈ one polymorphic virtual call
Queued send, one Group thread	~90 ns	queue push/pop + dispatch
Queued send, separate threads	~3,370 ns	cross-core mailbox wakeup, twice
Pooled round trip (2 messages)	~92 ns	vs ~128 ns with plain new
Allocator tail (max observed)	~33 μs	vs ~5.5 ms unpooled — the durable win

MEASURED Round trip through the framework — send, handle, reply — with no pipelining, from the microbenchmarks in `actors/cpp/perf`.

MEASURED End-to-end **tick-to-trade**, measured on the colocated stack, runs ~100 μs at the median with p99 under a millisecond — market data in to order on the wire, not a component benchmark.

SHADOW-POV EXECUTION AND SMART ORDER ROUTING

ES futures · 246 sessions of calendar 2025 · 100 contracts bought and 100 sold per window

On observing a third-party add, Shadow-POV places its own limit order at that order's price, on that order's venue, records nothing but the pair of exchange identifiers, and cancels when the order it follows ends. It is a low-latency, model-free execution algorithm with excellent slippage characteristics, and it carries a built-in smart order router that trades multiple venues by following the volume on each.

Relative slippage prices each leg against the mid at the instant the window opened and averages the two, so the market's drift during the window cancels.

	PASSIVE	AGGRESSIVE
ZERO SIMULATED LATENCY		
Completed windows	9,592	9,581
Relative slippage, ticks per contract	+0.0955 ± 0.0130	+0.0952 ± 0.0135
Aggregate participation	4.46 %	4.86 %
Time to fill 100	72.7 s	62.4 s
Filled on arrival rather than resting	1.04 %	51.86 %

MEASURED Intervals are 95 % and clustered by session. The passive mode rests for 99 % of its executed quantity and the aggressive mode crosses for half of it, at matched participation — and the round trip costs the same either way, to the fourth decimal, over more than nine thousand windows each.

What latency costs. The same delay applied to all three paths at once: the outbound order, the outbound cancel, and the inbound market-data feed.

	PASSIVE	AGGRESSIVE
DELAY ON ORDER, CANCEL AND FEED		
None	+0.097	+0.087
500 µs	+0.121	+0.176
1 ms	+0.132	+0.211
2.5 ms	+0.141	+0.240
5 ms	+0.150	+0.296

MEASURED Relative slippage in ticks per contract, same corpus. Passive loses about 0.011 ticks per contract per millisecond of delay and aggressive about 0.042 — roughly four times as fast — and the ordering between them reverses inside the first half-millisecond.